

Refactoring

Oliviero Riganelli

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public class Employee {  
  
    private final BigDecimal monltySalary = new BigDecimal(100.00);  
    private final BigDecimal bonus = new BigDecimal(20.00);  
    private final BigDecimal commission = new BigDecimal(10.00);  
  
    public BigDecimal getPaymentAmount(EmployeeType type) {  
  
        switch(type){  
            case BigDecimal ENGINEER;  
                return monltySalary;  
            case BigDecimal SALESMAN;  
                return monltySalary.add(commission);  
            case BigDecimal MANAGER;  
                return monltySalary.add(bonus);  
            default:  
                throw new RuntimeException("Incorrect Employee");  
        }  
    }  
}
```

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public class Employee {  
  
    private final BigDecimal monltySalary = new BigDecimal(100.00);  
    private final BigDecimal bonus = new BigDecimal(20.00);  
    private final BigDecimal commission = new BigDecimal(10.00);  
  
    public BigDecimal getPaymentAmount(EmployeeType type) {  
  
        switch(type){  
            case BigDecimal ENGINEER;  
                return monltySalary;  
            case BigDecimal SALESMAN;  
                return monltySalary.add(commission);  
            case BigDecimal MANAGER;  
                return monltySalary.add(bonus);  
            default:  
                throw new RuntimeException("Incorrect Employee");  
        }  
    }  
}
```

Switch Statement

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public interface Employee {  
  
    final BigDecimal monthlySalary = new BigDecimal(100);  
    BigDecimal getPaymentAmount();  
  
}
```

```
public class Engineer implements Employee {  
  
    public BigDecimal getPaymentAmount() {  
        return monthlySalary;  
    }  
  
}
```

```
public class Salesman implements Employee {  
  
    private BigDecimal commission = new BigDecimal(10);  
  
    public BigDecimal getPaymentAmount() {  
        return monthlySalary.add(commission);  
    }  
  
}
```

```
public class Manager implements Employee {  
  
    private BigDecimal bonus = new BigDecimal(20);  
  
    public BigDecimal getPaymentAmount() {  
        return monthlySalary.add(bonus);  
    }  
  
}
```

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public class DefaultListenableGraph<V, E> {  
    [...]  
    public E addEdge(V sourceVertex, V targetVertex) {  
        E e = super.addEdge(sourceVertex, targetVertex);  
        if (e != null) {  
            fireEdgeChange(e, ADDITION);  
        }  
        List<V> n1 = sourceVertex.getNeighbors();  
        List<V> n2 = targetVertex.getNeighbors();  
        List<V> toUpdate = new ArrayList<V>();  
        toUpdate.addAll(n1);  
        toUpdate.addAll(n2);  
        update(toUpdate);  
        return e;  
    }  
    [...]  
}
```

```
[..]  
public E removeEdge(V sourceVertex, V targetVertex) {  
    E e = super.removeEdge(sourceVertex, targetVertex);  
    if (e != null) {  
        fireEdgeChange(e, REMOVAL);  
    }  
    List<V> n1 = sourceVertex.getNeighbors();  
    List<V> n2 = targetVertex.getNeighbors();  
    List<V> toUpdate = new ArrayList<V>();  
    toUpdate.addAll(n1);  
    toUpdate.addAll(n2);  
    update(toUpdate);  
    return e;  
}  
[...]
```

Code Smell

Duplicated Code

```
public class DefaultListenableGraph<V, E> {  
    [...]  
    public E addEdge(V sourceVertex, V targetVertex) {  
        E e = super.addEdge(sourceVertex, targetVertex);  
        if (e != null) {  
            fireEdgeChange(e, ADDITION);  
        }  
        List<V> n1 = sourceVertex.getNeighbors();  
        List<V> n2 = targetVertex.getNeighbors();  
        List<V> toUpdate = new ArrayList<V>();  
        toUpdate.addAll(n1);  
        toUpdate.addAll(n2);  
        update(toUpdate);  
        return e;  
    }  
    [...]  
}
```

```
[..]  
public E removeEdge(V sourceVertex, V targetVertex) {  
    E e = super.removeEdge(sourceVertex, targetVertex);  
    if (e != null) {  
        fireEdgeChange(e, REMOVAL);  
    }  
    List<V> n1 = sourceVertex.getNeighbors();  
    List<V> n2 = targetVertex.getNeighbors();  
    List<V> toUpdate = new ArrayList<V>();  
    toUpdate.addAll(n1);  
    toUpdate.addAll(n2);  
    update(toUpdate);  
    return e;  
}  
[...]  
}
```

Refactoring

```
public class DefaultListenableGraph<V, E> {  
    [...]  
    private void fireAndUpdate(E e, int action, V sourceVertex, V targetVertex){  
        if (e != null) {  
            fireEdgeChange(e, action);  
        }  
        List<V> n1 = sourceVertex.getNeighbors();  
        List<V> n2 = targetVertex.getNeighbors();  
        List<V> toUpdate = new ArrayList<V>();  
        toUpdate.addAll(n1);  
        toUpdate.addAll(n2);  
        update(toUpdate);  
    }  
    [...]  
    public E addEdge(V sourceVertex, V targetVertex) {  
        E e = super.addEdge(sourceVertex, targetVertex);  
        fireAndUpdate(e, ADDITION, sourceVertex, targetVertex);  
        return e;  
    }  
    [...]  
    public E removeEdge(V sourceVertex, V targetVertex) {  
        E e = super.removeEdge(sourceVertex, targetVertex);  
        fireAndUpdate(e, REMOVAL, sourceVertex, targetVertex);  
        return e;  
    }  
    [...]  
}
```

Extract method

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public class Employee {  
    private Address officeAddress = null;  
    [...]  
    public String getMailingAddress() {  
        StringBuilder mailingAddress = new StringBuilder();  
        mailingAddress.append(officeAddress.BuildingName + " - " + officeAddress.Office);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.BuildingNumber + " " + officeAddress.StreetName);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.City + ", " + officeAddress.State);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.PostalCode);  
        return mailingAddress.toString();  
    }  
    [...]  
}
```


Code Smell

```
public class Employee {  
    private Address officeAddress = null;  
    [...]  
    public String getMailingAddress() {  
        StringBuilder mailingAddress = new StringBuilder();  
        mailingAddress.append(officeAddress.BuildingName + " - " + officeAddress.Office);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.BuildingNumber + " " + officeAddress.StreetName);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.City + ", " + officeAddress.State);  
        mailingAddress.append("\n");  
        mailingAddress.append(officeAddress.PostalCode);  
        return mailingAddress.toString();  
    }  
    [...]  
}
```

Feature Envy

Refactoring

```
public class Address {  
    [...]  
    public String toString() {  
        StringBuilder address = new StringBuilder();  
        address.append(BuildingName + " - " + Office);  
        address.append("\n");  
        address.append(BuildingNumber + " " + StreetName);  
        address.append("\n");  
        address.append(City + ", " + State);  
        address.append("\n");  
        address.append(PostalCode);  
        return address.toString();  
    }  
    [...]  
}
```

Move method

```
public class Employee {  
    private Address officeAddress = null;  
    [...]  
    public String getMailingAddress() {  
        return officeAddress.toString();  
    }  
    [...]  
}
```

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
class Figura {  
    enum Forma {  
        RETTANGOLO, CERCHIO  
    };  
    final Forma forma ;  
    double base ;  
    double altezza ;  
    double raggio ;  
    Figura ( double raggio ) {  
        forma = Forma.CERCHIO;  
        this.raggio = raggio ;  
    }  
}
```

```
Figura(double base, double altezza) {  
    forma = Forma.RETTANGOLO;  
    this.base = base ;  
    this.altezza = altezza ;  
}  
double area() {  
    switch ( forma ) {  
        case RETTANGOLO:  
            return base * altezza ;  
        case CERCHIO:  
            return Math.PI * (raggio * raggio);  
        default :  
            throw new AssertionError ( ) ;  
    }  
}  
}
```

Code Smell

```
class Figura {  
    enum Forma {  
        RETTANGOLO, CERCHIO  
    };  
    final Forma forma ;  
    double base ;  
    double altezza ;  
    double raggio ;  
    Figura ( double raggio ) {  
        forma = Forma.CERCHIO;  
        this.raggio = raggio ;  
    }  
}
```

```
Figura(double base, double altezza) {  
    forma = Forma.RETTANGOLO;  
    this.base = base ;  
    this.altezza = altezza ;  
}  
double area() {  
    switch ( forma ) {  
        case RETTANGOLO:  
            return base * altezza ;  
        case CERCHIO:  
            return Math.PI * (raggio * raggio);  
        default :  
            throw new AssertionError ( ) ;  
    }  
}
```

Switch Statement in *double area()*

Refactoring

```
abstract class Figura {  
    abstract double area();  
}  
  
class Rettangolo extends Figura {  
    final double base;  
    final double altezza;  
    Rettangolo(double base, double altezza) {  
        this.base = base;  
        this.altezza = altezza;  
    }  
    double area() {  
        return base * altezza;  
    }  
}
```

```
class Cerchio extends Figura {  
    final double raggio;  
    Cerchio ( double raggio ) {  
        this.raggio = raggio;  
    }  
    double area() {  
        return Math.PI * (raggio * raggio);  
    }  
}
```

Sostituire il Type Code con le sottoclassi
Sostituire switch con polimorfismo

Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
public class Disegno {  
    private List< Rettangolo > rettangoli;  
    private List< Cerchio > cerchi;  
  
    double area(Cerchio c) {return c.getRaggio() * c.getRaggio() * Math.PI;}  
    double area(Rettangolo r) {return r.getAltezza() * r.getLarghezza();}  
    double perimetro(Cerchio c) {return 2 * Math.PI * c.getRaggio();}  
  
    double perimetro(Rettangolo r) {  
        return 2 * (r.getAltezza() + r.getLarghezza());  
    }  
  
    public double area() {  
        double area = 0;  
        for (Rettangolo r : rettangoli) area += area(r);  
        for (Cerchio c : cerchi) area += area(c);  
        return area;  
    }  
  
    public double perimetro() {  
        double perimetro = 0;  
        for (Rettangolo r : rettangoli) perimetro += perimetro(r);  
        for (Cerchio c : cerchi) perimetro += perimetro(c);  
        return perimetro;  
    }  
}
```

```
class Cerchio {  
    private double raggio;  
  
    double getRaggio() {return raggio;}  
    void setRaggio(double raggio) {this.raggio = raggio;}  
}  
  
class Rettangolo {  
    private double larghezza, altezza;  
  
    double getLarghezza() {return larghezza;}  
    void setLarghezza(double larghezza) {this.larghezza = larghezza;}  
  
    double getAltezza() {return altezza;}  
    void setAltezza(double altezza) {this.altezza = altezza;}  
}
```

Code Smell

Feature Envy e Data class

```
public class Disegno {  
    private List< Rettangolo > rettangoli;  
    private List< Cerchio > cerchi;  
  
    double area(Cerchio c) {return c.getRaggio() * c.getRaggio() * Math.PI;}  
    double area(Rettangolo r) {return r.getAltezza() * r.getLarghezza();}  
    double perimetro(Cerchio c) {return 2 * Math.PI * c.getRaggio();}  
  
    double perimetro(Rettangolo r) {  
        return 2 * (r.getAltezza() + r.getLarghezza());  
    }  
  
    public double area() {  
        double area = 0;  
        for (Rettangolo r : rettangoli) area += area(r);  
        for (Cerchio c : cerchi) area += area(c);  
        return area;  
    }  
  
    public double perimetro() {  
        double perimetro = 0;  
        for (Rettangolo r : rettangoli) perimetro += perimetro(r);  
        for (Cerchio c : cerchi) perimetro += perimetro(c);  
        return perimetro;  
    }  
}
```

```
class Cerchio {  
    private double raggio;  
  
    double getRaggio() {return raggio;}  
    void setRaggio(double raggio) {this.raggio = raggio;}  
}  
  
class Rettangolo {  
    private double larghezza, altezza;  
  
    double getLarghezza() {return larghezza;}  
    void setLarghezza(double larghezza) {this.larghezza = larghezza;}  
  
    double getAltezza() {return altezza;}  
    void setAltezza(double altezza) {this.altezza = altezza;}  
}
```

Refactoring

```
public class Disegno {  
    private List< Rettangolo > rettangoli;  
    private List< Cerchio > cerchi;  
  
    double area(Cerchio c) {return c.getRaggio() * c.getRaggio() * Math.PI;}  
    double area(Rettangolo r) {return r.getAltezza() * r.getLarghezza();}  
    double perimetro(Cerchio c) {return 2 * Math.PI * c.getRaggio();}  
  
    double perimetro(Rettangolo r) {  
        return 2 * (r.getAltezza() + r.getLarghezza());  
    }  
  
    public double area() {  
        double area = 0;  
        for (Rettangolo r : rettangoli) area += area(r);  
        for (Cerchio c : cerchi) area += area(c);  
        return area;  
    }  
  
    public double perimetro() {  
        double perimetro = 0;  
        for (Rettangolo r : rettangoli) perimetro += perimetro(r);  
        for (Cerchio c : cerchi) perimetro += perimetro(c);  
        return perimetro;  
    }  
}
```

```
class Cerchio {  
    private double raggio;  
    double getRaggio() {return raggio;}  
    void setRaggio(double raggio) {this.raggio = raggio;}  
}
```

Move Method

Ulteriori raffinamenti:

Creare una interfaccia *Figura* con i metodi *area()* e *perimetro()* (Extract Interface) e cambiare la classe *Disegno* in modo che abbia una sola lista di figure e i metodi *area()* e *perimetro()* in modo che usino le versioni astratte dei metodi esposti della classe *Figura*.

```
double getAltezza() {return altezza;}  
void setAltezza(double altezza) {this.altezza = altezza;}  
}
```


Individuare e rimuovere uno o più code smell dal seguente frammento di codice.

```
int[] array1 = {0,1,2,3};
int[] array2 = {4,5,2,35};

int sum1 = 0;
int sum2 = 0;
int average1 = 0;
int average2 = 0;

for (int i = 0; i < array1.length ; i++)
{
    sum1 += array1[i];
}
average1 = sum1/array1.length;

for (int i = 0; i < array2.length; i++)
{
    sum2 += array2[i];
}
average2 = sum2/array2.length;
```

Code Smell

```
int[] array1 = {0,1,2,3};  
int[] array2 = {4,5,2,35};
```

```
int sum1 = 0;  
int sum2 = 0;  
int average1 = 0;  
int average2 = 0;
```

```
for (int i = 0; i < array1.length ; i++)  
{  
    sum1 += array1[i];  
}  
average1 = sum1/array1.length;
```

```
for (int i = 0; i < array2.length; i++)  
{  
    sum2 += array2[i];  
}  
average2 = sum2/array2.length;
```

Duplicated Code

Refactoring

```
int[] array1 = {0,1,2,3};  
int[] array2 = {4,5,2,35};
```

```
int average1 = calcAverage(array1);  
int avarage2 = calcAverage(array2);
```

...

```
public int calcAverage(int[] array) {  
    int sum = 0;  
    for (int i = 0; i < array.length; i++)  
    {  
        sum += array[i];  
    }  
    return sum/array.length;  
}
```

Extract Method